

Boolean Laws:

deal forms switch AND (0,1) with OR (+, v)

and constant 0's with 1's. for any expression

if 1 is 0 contains at least two elements, 0 and 1

so to 0 $\neq 1$ 2. Closures $a, b \in B \Rightarrow a + b \in B$

ii) $a \cdot b \in B$, 3. Commutativity $a, b \in B: a \cdot b = b \cdot a$

4. identities, $0 \cdot a = 0, 1 \cdot a = a$

ii) $a \cdot 1 = a \Rightarrow$ duals $a \cdot 0 = 0, a + 1 = 1$, So distrib

i) $a + (b \cdot c) = (a + b) \cdot (a + c)$ ii) $a \cdot (b + c) = a \cdot b + a \cdot c$

6. complement $a + \bar{a} = 1, a \cdot \bar{a} = 0$

7. idempotence: $a + a = a$, duals $a \cdot a = a$, involution:

$(\bar{\bar{a}}) = a$ (not not!), 9. associativity: $(a + b) + c = a + (b + c)$

$= a + (b + c) = a + b + c$, duals $(a \cdot b) \cdot c = a \cdot (b \cdot c)$

$= a \cdot b \cdot c$, Simplification Theorems: 10. $a \cdot b + a \cdot \bar{b} = a$

duals: $(a + b) \cdot (a + \bar{b}) = a$, 11. $a + a \cdot b = a$, duals $a \cdot a + b = a$

$(a + b) = a$, 12. $(a + \bar{b}) \cdot b = a \cdot b$, duals $(a \cdot \bar{b}) + b = a + b$

13. $(a + b + \bar{a} + \bar{b}) = 1$, duals $(a \cdot b + \bar{a} + \bar{b}) = 1$

Sum of Products $\hookrightarrow$ DNF from DM procedure: Find

all terms that output 1, connect with "1"s, then connect

implicants with "1"s, each implicant is called a

minterm, notation: $\Sigma m(0, 6, 7) = \text{IIM}(1, 2, 3, 4, 5)$

$\Rightarrow$ complement of expression: $\Sigma m(1, 2, 3, 4, 5)$, Product

Of Sums $\hookrightarrow$ CNF from DM procedure: Find all

terms that output 0, create

implicants by connecting terms with

"1"s, then connect implicants with

"0"s $\rightarrow$ implicants here are called

maxterms

Timing $\rightarrow$ STC

Setup Time Constraint: $T_C \geq$

$t_{\text{req}} + t_{\text{pd}} + t_{\text{setup}}$ or $t_{\text{pd}} \leq$

$T_C - (t_{\text{req}} + t_{\text{setup}}) \rightarrow \text{HTC}$

Hold Time Constraint:

$t_{\text{req}} + t_{\text{hd}} \geq \text{thold}$, min-delay

constraint: $t_{\text{hd}} \geq \text{thold} - t_{\text{req}}$

STC + skew: $t_{\text{pd}} \leq T_C - (t_{\text{req}} + t_{\text{setup}} + t_{\text{skew}})$, HTC + skew:

$t_{\text{req}} + t_{\text{hd}} \geq \text{thold} + t_{\text{skew}}$

min-delay + skew: $t_{\text{hd}} \geq \text{thold} + t_{\text{skew}}$

$t_{\text{skew}} - t_{\text{req}}$, Critical Path:

Σ propagation delays, Short Path:

Σ contamination delays, Fast:

STC violations fixable by

increasing T_C , HTC not

Verilog: used for slicing

Syntax: $[] \rightarrow$ bit-select

() $\rightarrow$ parentheses, ! $\rightarrow$ NOT / logical negation, ①

$\sim \rightarrow$ NOT, & $\rightarrow$ AND, $\rightarrow$ OR, $\wedge \rightarrow$ NAND,

$\vee \rightarrow$ NOR, $\wedge \rightarrow$ XOR, $\wedge \rightarrow$ XNOR, 2,

vars. can be 0, 1, 2, x, can be used

also arithmetic/ternary identical unary and binary

to Java, simple C: continuous assign

(concurrent), Numbers: N (6 value,

base: 'b' $\rightarrow$ binary, 'o' $\rightarrow$ octal, 'x' $\rightarrow$ size in bits

12 decimal, 'h' $\rightarrow$ hex, physical instantiation:

Module name (input a, b, c, output y),

end with "endmodule", inputs/outputs can

be declared inside module too,

concatenation: assign c = {a[1:0], b[3:1]}

this creates a 2+6-wide bus

multiple wires, wires are used sort of

like intermediate variables, regular

as CL blocks use always @(*) and usually

= to assign $\rightarrow$ blocking/parallel, SL uses

always @(posedge clk) and $\leq$ to assign

$\rightarrow$ non-blocking sequential, high reset

async includes possible reset in senselist,

low reset: async, analogs, case

checks value of input in base 10, used

for an input data use case (data), fish!

DDCA Client Sheet

FFK birdlaustis

Cache formulas

Hit Rate = $(\#hits) / (\#hits + \#misses)$
(low many misses)

= $(\#hits) / (\#accesses)$, $A = \text{assoc.}$, $B = \text{block size}$

In bytes, $C = \text{cache capacity in bytes} \rightarrow N_{\text{sets}} = \frac{C}{B}$

$CBA, N_{\text{blocks}} = C/B$, offset = $\log_2 B$, index = $\log_2 N_{\text{sets}}$

tag = $A \text{ bits} - \text{index} - \text{offset}$, $AMAT = \frac{1}{\text{Hit Rate}} + \frac{\text{Miss Penalty}}{\text{Hit Rate}}$

+ Miss Rate + Miss Penalty, miss reasons: compulsory, capacity/conflict, associativity

DM $\rightarrow$ 1 way/set, set assoc. $\rightarrow$ $\frac{1}{\text{way size}}$

A way/set, fully assoc. $\rightarrow$ 1 set, write back $\rightarrow$ dirty bit

on execution, write through & immediately write next level / LRU

avoid least recent

VMem: offset = $\log_2(\text{page size})$, $\frac{\text{VMem}}{\text{page size}}$ # (Virtual pages)

= $(\text{VMem size}) / (\text{page size})$, # (bits for p.pn) = $\log_2 \frac{\text{VMem}}{\text{page size}}$

(#pages), $\text{VPN} = \text{VA}_{\text{bits}} - \text{offset}$, $\text{PPN} = \text{PA}_{\text{bits}} - \text{offset}$

(page table entries, level 1) = $2^{\text{VPN bits}}$, $\text{TLB reach} = \text{entries}$

page size, $\text{VA} = \lfloor \text{VPN} \rfloor \cdot \text{offset} + \text{Translation PA} = \lfloor \text{PPN} \rfloor \cdot \text{offset}$

mixed page-size block: $\min = \#(\text{entries})$ & smallest page

$\max = \#(\text{entries})$ & largest page

Prefetching

Accuracy = $\#(\text{prefetches}) / \#(\text{prefetches issued})$

$\text{Coverage} = \frac{\#(\text{prefetches})}{\#(\text{misses})}$ (best case)

$\text{Coverage} = \frac{\#(\text{prefetches})}{\#(\text{misses})}$ (best case)

(on-time prefetches) / # (all prefetches), Bandwidth

Consumption: mem. bandwidth consumed with prefetcher

mem. bandwidth consumed without prefetcher

$\rightarrow$ exam uses prefetchers in steady state

if outstanding req. to X exists, later prefetch

coalesces $\rightarrow$ not extra DRAM req. can't unpin

mem requests, stride prefetching: $A, A+N,$

$A+2N, \dots$, next N : $A+1, \dots, A+N$

SIMD Vector: $\frac{\#(\text{threads})}{\text{way size}}$

capacity/conflict, associativity: DM $\rightarrow$ 1 way/set, set assoc. $\rightarrow$ $\frac{1}{\text{way size}}$

A way/set, fully assoc. $\rightarrow$ 1 set, write back $\rightarrow$ dirty bit

on execution, write through & immediately write next level / LRU

avoid least recent

VMem: offset = $\log_2(\text{page size})$, $\frac{\text{VMem}}{\text{page size}}$ # (Virtual pages)

= $(\text{VMem size}) / (\text{page size})$, # (bits for p.pn) = $\log_2 \frac{\text{VMem}}{\text{page size}}$

(#pages), $\text{VPN} = \text{VA}_{\text{bits}} - \text{offset}$, $\text{PPN} = \text{PA}_{\text{bits}} - \text{offset}$

(page table entries, level 1) = $2^{\text{VPN bits}}$, $\text{TLB reach} = \text{entries}$

page size, $\text{VA} = \lfloor \text{VPN} \rfloor \cdot \text{offset} + \text{Translation PA} = \lfloor \text{PPN} \rfloor \cdot \text{offset}$

mixed page-size block: $\min = \#(\text{entries})$ & smallest page

$\max = \#(\text{entries})$ & largest page

Accuracy = $\#(\text{prefetches}) / \#(\text{prefetches issued})$

Coverage = $\frac{\#(\text{prefetches})}{\#(\text{misses})}$ (best case)

Coverage = $\frac{\#(\text{prefetches})}{\#(\text{misses})}$ (best case)

(on-time prefetches) / # (all prefetches), Bandwidth

Consumption: mem. bandwidth consumed with prefetcher

but stalls can repeatedly hit some banks

Examples

Problem Description (PD): Considers MIPS CPU

with 5 pipeline stages and 1 ALU,

F/D, EXE, MEM, WB; all take 1 cycle

The CPU has data forwarding and branch pred.

after finding flow dep. operand is forwarded from

MEM or WB to EXE, can only be forwarded

from WB, the branch pred. is always taken / PC

addr. available after D & Internal RF Pred

is available $\rightarrow$ ins. writes into reg. in first half of cycle

other ins. can access $\#(\text{reg.})$ in second half, assume

in-order exec.

Consider the following code:

loop: lw \$2, 0(\$1) #load val. from addr. in \$1 to \$2

and \$2, \$2, \$2 #1 # bitwise and, \$2 & #1

begz \$2, 0(\$1) #load val. from \$1 to \$2

add \$4, \$4, \$3 #increment \$4 by 4

bne \$1, \$5, loop

$\rightarrow$ assume \$1 = 0x100, \$4 = 0, \$5 = 0x110

Mem. Addr. 0x100 0x104 0x108 0x10C 0x110

Value 0x2 0x4 0x6 0x8 0xA

0x12 0x14 0x16 0x18 0x1A

0x1C 0x1E 0x20 0x22 0x24

0x26 0x28 0x2A 0x2C 0x2E

0x30 0x32 0x34 0x36 0x38

0x3A 0x3C 0x3E 0x40 0x42

0x44 0x46 0x48 0x4A 0x4C

0x4E 0x50 0x52 0x54 0x56

0x58 0x5A 0x5C 0x5E 0x60

0x62 0x64 0x66 0x68 0x6A

0x6C 0x6E 0x70 0x72 0x74

0x76 0x78 0x7A 0x7C 0x7E

0x80 0x82 0x84 0x86 0x88

0x8A 0x8C 0x8E 0x90 0x92

0x94 0x96 0x98 0x9A 0x9C

0x9E 0xA0 0xA2 0xA4 0xA6

0xA8 0xAA 0xAC 0xAE 0xB0

0xB2 0xB4 0xB6 0xB8 0xBA

0xBC 0xBE 0xC0 0xC2 0xC4

0xC6 0xC8 0xCA 0xCC 0xCE

0xD0 0xD2 0xD4 0xD6 0xD8

0xDA 0xDC 0xDE 0xE0 0xE2

0xE4 0xE6 0xE8 0xEA 0xEC

0xEE 0xF0 0xF2 0xF4 0xF6

0xF8 0xFA 0xFC 0xFE 0x100

0x102 0x104 0x106 0x108 0x10A

0x10C 0x10E 0x110 0x112 0x114

0x116 0x118 0x11A 0x11C 0x11E

0x120 0x122 0x124 0x126 0x128

0x12A 0x12C 0x12E 0x130 0x132

0x134 0x136 0x138 0x13A 0x13C

0x13E 0x140 0x142 0x144 0x146

0x148 0x14A 0x14C 0x14E 0x150

0x152 0x154 0x156 0x158 0x15A

0x15C 0x15E 0x160 0x162 0x164

0x166 0x168 0x16A 0x16C 0x16E

0x170 0x172 0x174 0x176 0x178

0x17A 0x17C 0x17E 0x180 0x182

0x184 0x186 0x188 0x18A 0x18C

0x18E 0x190 0x192 0x194 0x196

0x198 0x19A 0x19C 0x19E 0x1A0

0x1A2 0x1A4 0x1A6 0x1A8 0x1AA

0x1AC 0x1AE 0x1B0 0x1B2 0x1B4

0x1B6 0x1B8 0x1BA 0x1BC 0x1BE

0x1C0 0x1C2 0x1C4 0x1C6 0x1C8

0x1CA 0x1CC 0x1CE 0x1D0 0x1D2

0x1D4 0x1D6 0x1D8 0x1DA 0x1DC

0x1DE 0x1E0 0x1E2 0x1E4 0x1E6

0x1E8 0x1EA 0x1EC 0x1EE 0x1F0

0x1F2 0x1F4 0x1F6 0x1F8 0x1FA

0x1FC 0x1FE 0x200 0x202 0x204

0x206 0x208 0x20A 0x20C 0x20E

0x210 0x212 0x214 0x216 0x218

0x21A 0x21C 0x21E 0x220 0x222

0x224 0x226 0x228 0x22A 0x22C

0x22E 0x230 0x232 0x234 0x236

0x238 0x23A 0x23C 0x23E 0x240

0x242 0x244 0x246 0x248 0x24A

0x24C 0x24E 0x250 0x252 0x254

0x256 0x258 0x25A 0x25C 0x25E

0x260 0x262 0x264 0x266 0x268

0x26A 0x26C 0x26E 0x270 0x272

0x274 0x276 0x278 0x27A 0x27C

0x27E 0x280 0x282 0x284 0x286

0x288 0x28A 0x28C 0x28E 0x290

0x292 0x294 0x296 0x298 0x29A

0x29C 0x29E 0x2A0 0x2A2 0x2A4

0x2A6 0x2A8 0x2AA 0x2AC 0x2AE

0x2B0 0x2B2 0x2B4 0x2B6 0x2B8

0x2BA 0x2BC 0x2BE 0x2C0 0x2C2

0x2C4 0x2C6 0x2C8 0x2CA 0x2CC

0x2CE 0x2D0 0x2D2 0x2D4 0x2D6

0x2D8 0x2DA 0x2DC 0x2DE 0x2E0

0x2E2 0x2E4 0x2E6 0x2E8 0x2EA

0x2EC 0x2EE 0x2F0 0x2F2 0x2F4

0x2F6 0x2F8 0x2FA 0x2FC 0x2FE

0x300 0x302 0x304 0x306 0x308

0x30A 0x30C 0x30E 0x310 0x312

0x314 0x316 0x318 0x31A 0x31C

0x31E 0x320 0x322 0x324 0x326

0x328 0x32A 0x32C 0x32E 0x330

0x332 0x334 0x336 0x338 0x33A

0x33C 0x33E 0x340 0x342 0x344

0x346 0x348 0x34A 0x34C 0x34E

0x350 0x352 0x354 0x356 0x358

0x35A 0x35C 0x35E 0x360 0x362

0x364 0x366 0x368 0x36A 0x36C

0x36E 0x370 0x372 0x374 0x376

0x378 0x37A 0x37C 0x37E 0x380

0x382 0x384 0x386 0x388 0x38A

0x38C 0x38E 0x390 0x392 0x394

0x396 0x398 0x39A 0x39C 0x39E

0x3A0 0x3A2 0x3A4 0x3A6 0x3A8

0x3AA 0x3AC 0x3AE 0x3B0 0x3B2

0x3B4 0x3B6 0x3B8 0x3BA 0x3BC

0x3BE 0x3C0 0x3C2 0x3C4 0x3C6

0x3C8 0x3CA 0x3CC 0x3CE 0x3D0

0x3D2 0x3D4 0x3D6 0x3D8 0x3DA

0x3DC 0x3DE 0x3E0 0x3E2 0x3E4

0x3E6 0x3E8 0x3EA 0x3EC 0x3EE

0x3F0 0x3F2 0x3F4 0x3F6 0x3F8

0x3FA 0x3FC 0x3FE 0x400 0x402

0x404 0x406 0x408 0x40A 0x40C

0x40E 0x410 0x412 0x414 0x416

0x418 0x41A 0x41C 0x41E 0x420

0x422 0x424 0x426 0x428 0x42A

0x42C 0x42E 0x430 0x432 0x434

0x436 0x438 0x43A 0x43C 0x43E

0x440 0x442 0x444 0x446 0x448

0x44A 0x44C 0x44E 0x450 0x452

0x454 0x456 0x458 0x45A 0x45C

0x45E 0x460 0x462 0x464 0x466

0x468 0x46A 0x46C 0x46E 0x470

0x472 0x474 0x476 0x478 0x47A

0x47C 0x47E 0x480 0x482 0x484

0x486 0x488 0x48A 0x48C 0x48E

0x490 0x492 0x494 0x496 0x498

0x49A 0x49C 0x49E 0x4A0 0x4A2

0x4A4 0x4A6 0x4A8 0x4AA 0x4AC

0x4AE 0x4B0 0x4B2 0x4B4 0x4B6

0x4B8 0x4BA 0x4BC 0x4BE 0x4C0

0x4C2 0x4C4 0x4C6 0x4C8 0x4CA

0x4CC 0x4CE 0x4D0 0x4D2 0x4D4

0x4D6 0x4D8 0x4DA 0x4DC 0x4DE

0x4E0 0x4E2 0x4E4 0x4E6 0x4E8

0x4EA 0x4EC 0x4EE 0x4F0 0x4F2

0x4F4 0x4F6 0x4F8 0x4FA 0x4FC

0x4FE 0x500 0x502 0x504 0x506

0x508 0

- The 2nd diagram is more intuitive
- branch misprediction; condition

$\Rightarrow$ answer, $29 + 15 = 36$ total cycles

nearby
\$5 = \$1
and depends

do you have the following

$$A, A+1, A+2, A+3, A+4, A+5, A+6,$$

→ if there is a req. for a cack block that

will be merged in the MSP and not go to

list 3 cache block address $\rightarrow$ if there is a

issues prefetch to cache block using that stride

⇒ what is the coverage of the prefecture?

After seeing 3 cache blocks in a row and prefetching

The fourth $\rightarrow$ these blocks are always useless and never

~~all~~ blocks are fetched by a new detector with $2/3$ coverage and

seeing a req. to M , and after ~~seeing the req. to M~~ seeing the req. to Z_2 .

infer memo. rego, preferen next 2 blocks)

next 2 are fetched, from $A+B$, we save one reg. to $A+9$, from $A+9$ we fetch $A+10$

e) what is the min. value of N for 100% coverage on

neg to B is seen, it can issue prefetch for 28

one block to $d \rightarrow$ in steady state, everything is detached

~~fix μ & fix α~~ quick note: $\mu = 2^{10}$ bits, $\mu b = 2^{20}$ bits

observing the

1, prefetcher
" " "
branch unit, can only execute assembly ops of Table 1,
- exec units / 1 store unit / 1 exec unit / 1 branch unit /

constant stroke
 $r_{D_{1-}}, r_{D_{11}}$, which are read at $r_{D_{1-}}$ and $r_{D_{11}}$ with $r_{D_{1-}}$ and $r_{D_{11}}$ filled in red.

memory word-address (1 word = 4 bytes)

6

FU type | **ins**

load $rdst \left[r_{s1}, r_{s2}, \#offset \right]$
 $load_inc \left[r_{dst}, r_{s1}, r_{s2}, \#offset \right]$

store $\left[r_{s1}, r_{s2}, \#offset \right], r_{d3}$
 $store_inc \left[r_{s1}, r_{s2}, \#offset \right], r_{d3}$

add $rdst, r_{s1}, r_{s2}$

mult
 $mult \left[r_{dst}, r_{s1}, r_{s2} \right]$

branch
 $bne \left[r_{s1}, \#offset, TARGET \right]$

any
 NOP

Class assembly code, N is even, pass, regs = r29, r30, r31, r26
 C-code maps A, B, C, D, r0 init with 0, r1 init with 1

C-code: (int 4 bytes long) Assembly code

```

int A[N+1];
int B[N+1];
int C[N+1];
int D[N+1];
%0 = init to 0, A, B, 0
for (int i = 0; i < N; i++)
  C[i] = (A[i] + B[i]) * D[i];
  
```

LOOP:

V_0 load $r_2, [r_{29}, r_{0}, \#0]$ $\%r_2 = a$
 V_1 load $r_3, [r_{30}, r_{0}, \#0]$ $\%r_3 = b$
 V_2 load $r_4, [r_{26}, r_{0}, \#0]$ $\%r_4 = c$
 V_3 load $r_5, [r_{26}, r_{0}, \#0]$ $\%r_5 = d$
 V_4 mult r_6, r_4, r_5 $\%r_6 = (c * d)$
 V_5 store $inc, [r_{31}, r_{0}, \#0], r_6$ $\%store = r_6$
 V_6 bne $r_{0}, \#N, LOOP$ $\%bne \text{ correct } i = N$

a) fill in this table with statically scheduled VLIW ins.

VLIW ins	load	store	mult	add	branch
1 LOOP	V_1	V_2	V_3	V_4	V_5
2	V_1	V_2	V_3	V_4	V_5
3	V_1	V_2	V_3	V_4	V_5
4	V_1	V_2	V_3	V_4	V_5
5	V_1	V_2	V_3	V_4	V_5

b) ratio between # useful ops and VLIW ins in code? $7/5 \rightarrow$ assembly ops (not NOP), 5 VLIW ins.

c) execution time $\leq N$ cycles $\rightarrow$ 1 iter. takes 5 cycles, there are 10 iterations

d) ~~Find~~ Compute the energy consumption of the sequence of their

The VLIW table $\rightarrow E = \text{power (avg)} \times \text{time}$, a useful op. consumes 5W, NOP 1W

$\text{freq.} = 1 \text{ GHz} \rightarrow \text{avg } T_c = \frac{1}{\text{freq.}} = 10^{-9} \text{ s}$ for NOP $(1 \text{ MHz} \rightarrow 1 \mu\text{s} = 10^{-6} \text{ s}) \rightarrow$

$\text{Power}_{AVG} = \frac{7 \times 6 \text{ W} + 23 \times 1 \text{ W}}{5} = 5.8 \text{ W}$, Time = No cycles $\times$ time = $N \times 5 \text{ ns}$

$\Rightarrow \text{Power}_{AVG} \times \text{Time} = 11.6 \text{ W} \times 5 \times 10^{-5} = 58.0 \text{ nJ}$

HW 4: 2007 FSC2 $\rightarrow$ Devs Engg. Transisto.

Consider this OoD machine frontend has 1 cycle FETCH / cycle DEC, EXEC, only R-type ins. dispatches 1 ins. into RS per cycle, in PO, dispatched in DEC, ins. always allocates first RS in top $\rightarrow$ bottom order at required FU, when ins. in RS finishes RS cleared, adder/multiplier not pipelined (take 2/3 cycles, RS of add. ~~multiplier~~ multi. is broadcast to RS entries (RAT in WB $\rightarrow$ dup. ins. can begin next cycle ~~multiplier~~ if all operands available in RS entry, when multiple ins. are ready to execute at FU in same cycle, oldest ready ins. is executed first, machine empty initially,

state of final dispatch

RAT		RS-add		RS-mult	
Ins	Val	Ins	Val	Ins	Val
A	0	A	0	A	0
B	0	B	0	B	0
C	0	C	0	C	0

c) give the four ins. dispatched into machine in PO (source regs. in either order)

1. MUL X, R1, R1, 2. ADD R2, X, R2, 3. ADD R4, R1, R1, 4. MUL R3, R1, R4, x.e. $R_1, R_3, R_4, 3$

b) assume machine flushes all ins from pipeline, restarts fetch from 1st ins, full pipeline

Cycle:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
MUL R1 ← R1, R1	ADD R2 ← R1, R1	ADD R4 ← R1, R1	MUL R3 ← R1, R4												

e) show state of RAT/RS after cycle 12:

RAT		RS-add		RS-mult	
Ins	Val	Ins	Val	Ins	Val
A	0	A	0	A	0
B	0	B	0	B	0
C	0	C	0	C	0

Method: compare with given RS/RAT, progress the arithmetic